

VIT[®]
BHOPAL



BMI & HEALTHY ROUTINE ASSISTANT

Submitted by	Abhinav Gupta
Registration Number	26BEC10049
Academic Year	2026-2027
Slot	B21+B22+B23+E14+E21+E22

INTRODUCTION:

BODY MASS INDEX TABLE

UNDERWEIGHT	NORMAL WEIGHT	OVERWEIGHT	OBESE	MORBIDLY OBESE
< 18.5	18.5 - 24.9	25.0 - 29.9	30.0 - 39.9	≥ 40.0
LOW	IDEAL	HIGH	VERY HIGH	EXTREMELY HIGH

The BMI & Healthy Routine Assistant is a beginner-friendly Python project built to calculate Body Mass Index (BMI) and provide general educational guidance on food, healthy daily routines, and the meaning of BMI as a health metric. The project is implemented as a menu-driven console application, structured into independent Python modules for clarity and maintainability.

The application allows the user to enter their weight and height and calculates BMI using the standard BMI formula. Based on the calculated BMI, the application displays a general BMI category and neutral feedback.

The system also provides general balanced food suggestions, a healthy daily routine, and educational information about BMI and its limitations.

PROBLEM STATEMENT:

Many people want a simple way to calculate BMI and understand basic information related to healthy routines. However, beginners may find health-related information difficult to understand.

The purpose of this project is to develop a simple menu-driven Python application that calculates BMI and provides general educational information about balanced food choices, daily routines, and BMI.

The application is designed for educational purposes and does not provide medical diagnosis or personalized medical advice.

The information use in this project is based on WHO (World Health Organization) Released data related to BMI.

OBJECTIVE:

->The main objectives of the project are:

- To calculate BMI using the user's weight and height.
- To display a general BMI category based on the calculated value.
- To provide general balanced food suggestions.
- To provide general healthy daily routine guidance.
- To explain BMI and its limitations.
- To demonstrate modular programming using Python.
- To implement input validation and basic error handling.
- To test the application using different valid and invalid inputs.

Functional Requirements:

- BMI Calculation

Input: Weight in kilograms, Height in centimetres

Processing: The program converts height from centimetres to metres and calculates BMI.

Output: BMI value, General BMI category, Neutral feedback.

- Food Suggestions: The system provides general balanced meal suggestions based on the BMI category obtained from the BMI calculation.
- Healthy Daily Routine The system provides general suggestions for: Morning, Daytime, Evening, Night .
- Learn About BMI The system explains: Meaning of BMI, BMI formula Purpose of BMI, Limitations of BMI.

- Main Menu and Program Control

The main program provides a menu through which the user can select different features and exit the application.

NON-FUNCTIONAL REQUIREMENTS:

- Usability
The application uses a simple menu-driven interface so that users can select features using numbered options.
- Reliability
The program handles invalid numerical inputs and non-positive values without terminating unexpectedly.
- Maintainability
The application is divided into separate Python modules. Each module is responsible for a specific functionality, making the code easier to understand and modify.
- Error Handling
The application uses input validation and exception handling to handle invalid user input.
- Performance
BMI calculations and information display are performed quickly because the application uses simple calculations and does not require external services or a database.

SYSTEM ARCHITECTURE

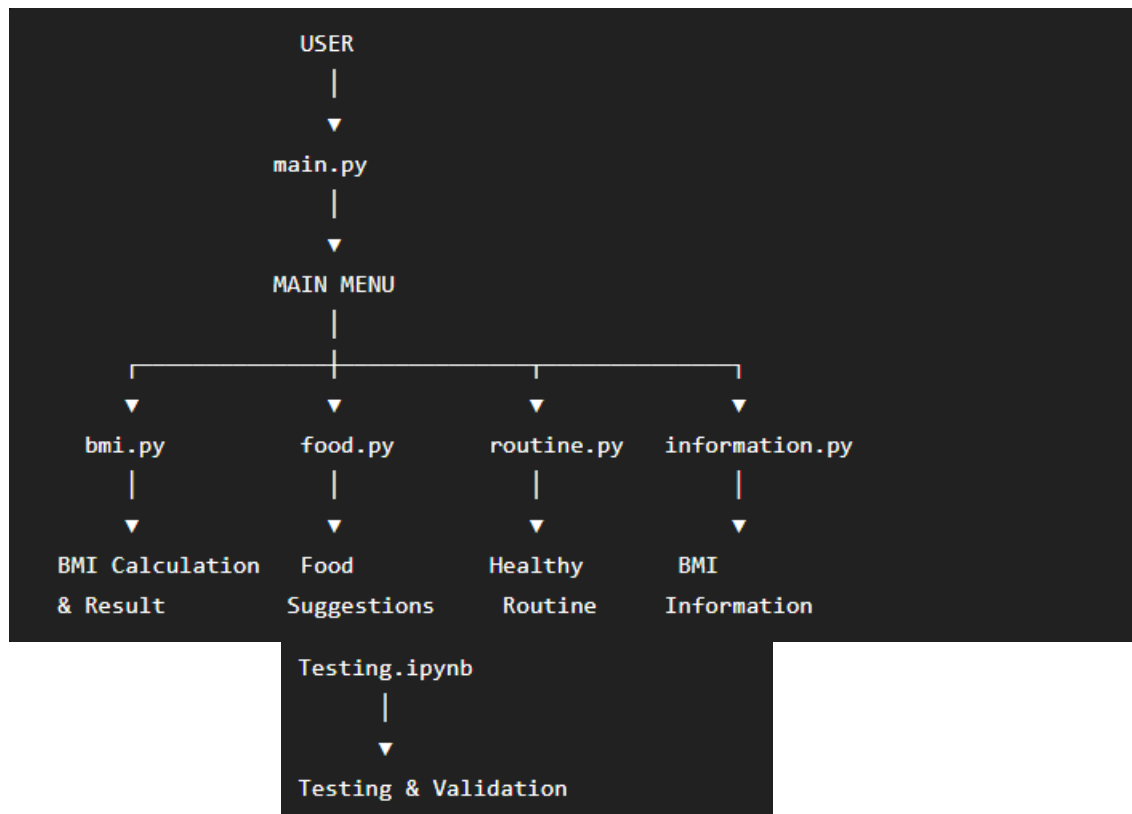
The application follows a simple modular architecture.

The main.py module acts as the main controller and displays the menu. Based on the user's selection, it calls the appropriate functional module.

Components

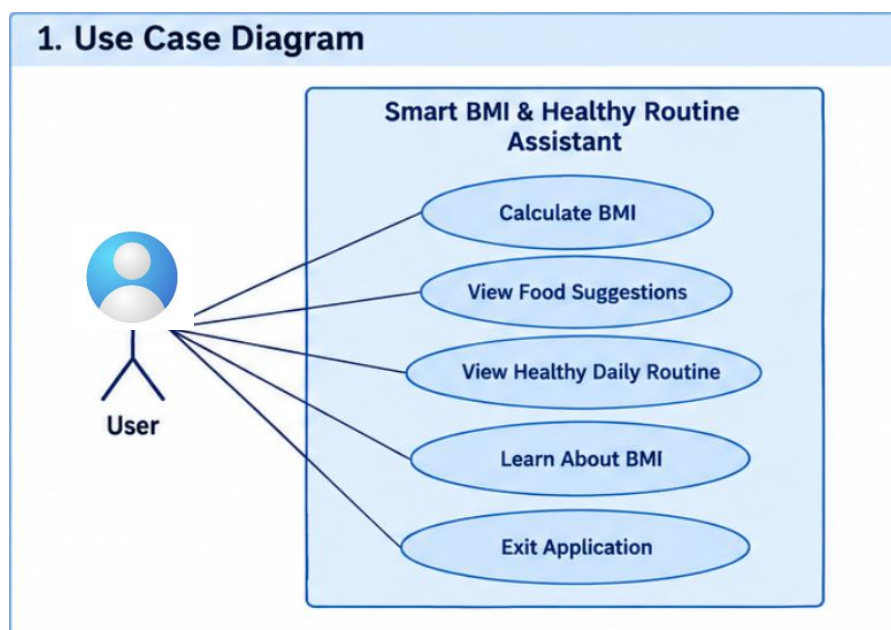
- main.py — Main menu and program control
- bmi.py — BMI calculation and input validation
- food.py — Food suggestions
- routine.py — Healthy daily routine
- information.py — BMI educational information
- Testing.ipynb — Testing and validation

SYSTEM ARCHITECTURE

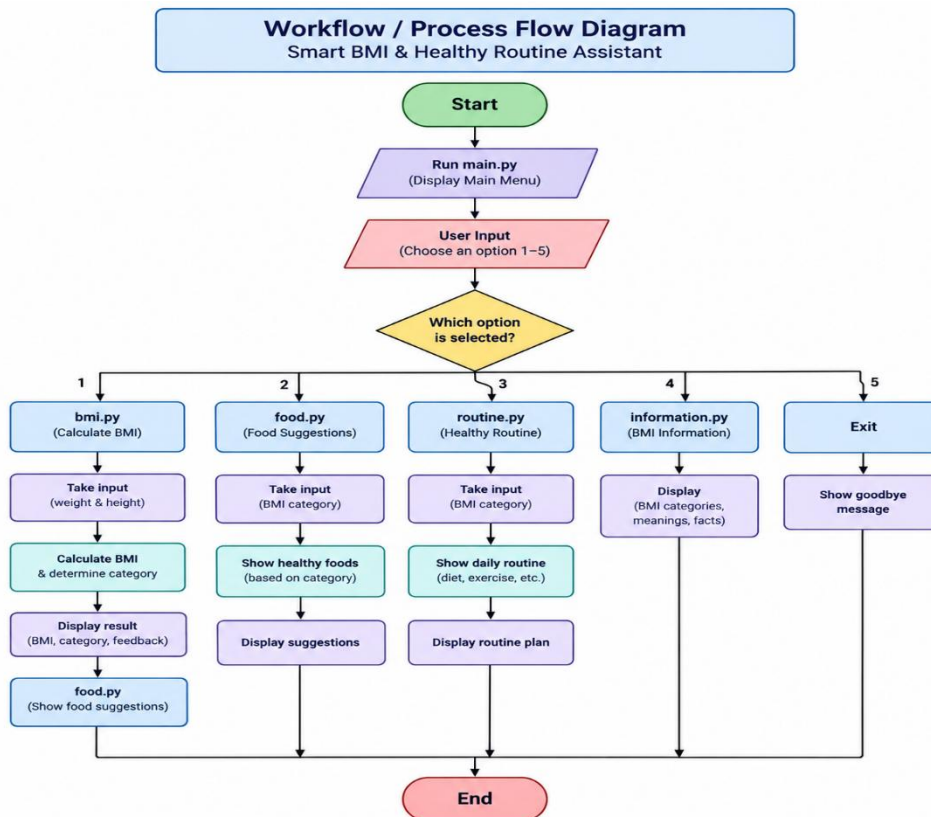


DESIGN DIAGRAMS

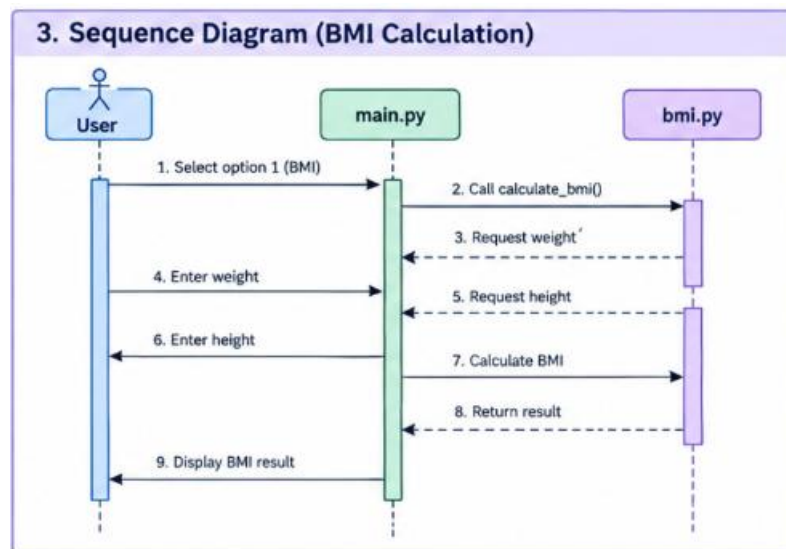
1. Use Case Diagram



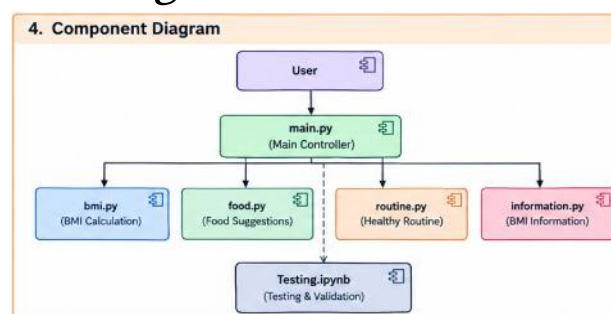
2. Workflow Diagram



3. Sequence Diagram



4. Class/Component Diagram



Design Decisions & Rationale

- Modular Structure

The project is divided into multiple Python files so that each major function has its own module. This improves organization and maintainability.

- Menu-Driven Interface

A numbered menu was selected because it provides a simple interaction method for a beginner-friendly command-line application.

- Input Validation

Input validation was included to prevent invalid values from causing program errors.

- No Database

The current application does not store user information or history. Therefore, a database is not required.

- Command-Line Execution

The application can be executed directly through the terminal using:

```
python main.py
```

This allows the project to run without requiring a graphical interface.

IMPLEMENTATION DETAILS

The project is implemented in Python using a modular structure.

main.py

Controls the main menu and connects the different modules.

bmi.py

Accepts weight and height, validates the input, calculates BMI, and displays the category and feedback.

food.py

Displays general balanced food suggestions based on the BMI category returned by the BMI module.

routine.py

Displays general healthy daily routine information.

information.py

Provides educational information about BMI, its formula, and its limitations.

Testing.ipynb

Used to test the application with different inputs and verify the expected outputs.

SCREENSHOTS / RESULTS

Screenshot 1 — BMI Calculation

```
jupyter Testing Last Checkpoint: yesterday
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

would you like to see food suggestions?

[*]: import main

=====
SMART BMI & HEALTHY ROUTINE ASSISTANT
=====
1. Calculate BMI
2. Food Suggestions
3. Healthy Daily Routine
4. Learn About BMI
5. Exit

Enter your choice: 1

--- BMI CALCULATOR ---
Enter weight in kg: 74
Enter height in cm: 181

Your BMI is: 22.59
Category: Usual adult BMI range
Feedback: Good result

Would you like to see food suggestions?
Enter yes or no:
```

Screenshot 2 — Food Suggestions

```
[*]: import main

=====
SMART BMI & HEALTHY ROUTINE ASSISTANT
=====
1. Calculate BMI
2. Food Suggestions
3. Healthy Daily Routine
4. Learn About BMI
5. Exit

Enter your choice: 1

--- BMI CALCULATOR ---
Enter weight in kg: 74
Enter height in cm: 181

Your BMI is: 22.59
Category: Usual adult BMI range
Feedback: Good result

Would you like to see food suggestions?
Enter yes or no: yes

--- FOOD SUGGESTIONS ---
Suggested balanced meals:
• Breakfast: Idli with sambar or oats with fruit
• Lunch: Roti, dal and vegetables
• Snack: Fruit or roasted chana
• Dinner: Roti, vegetables and dal
```

Screenshot 3 — Healthy Daily Routine

```
--- HEALTHY DAILY ROUTINE ---

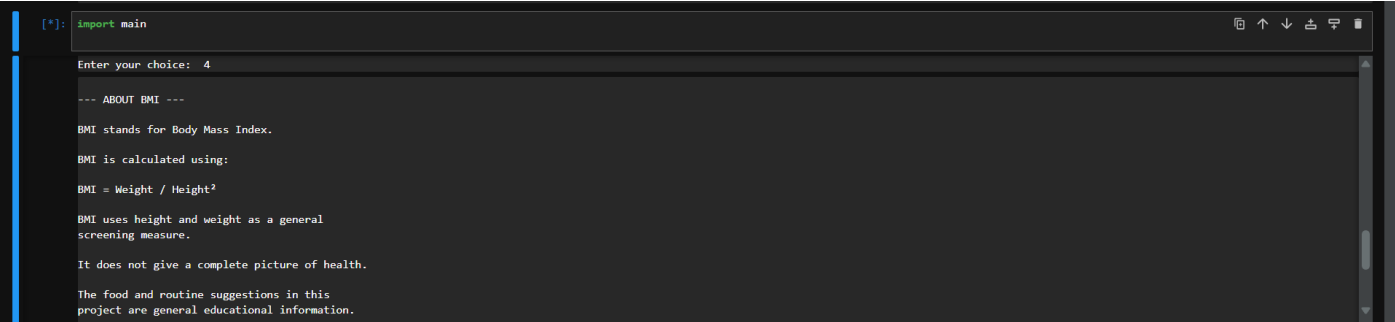
Morning:
• Start your day at a regular time.
• Drink water regularly.
• Have a balanced breakfast.

Daytime:
• Take regular movement breaks.
• Include fruits and vegetables in meals.

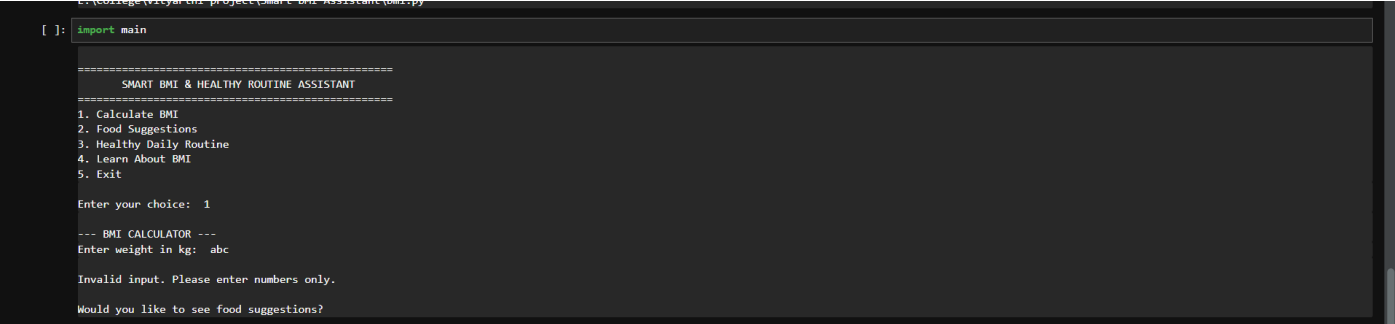
Evening:
• Include some age-appropriate physical activity.
• Have a balanced meal.

Night:
• Follow a regular sleep schedule.
• Reduce unnecessary screen use before sleeping.
```

Screenshot 4 — Learn About BMI



Screenshot 5 — Error Handling



12. TESTING APPROACH

The application was tested using both valid and invalid inputs.

Test	Input	Expected Result
Good BMI	74Kg, 181cm	BMI calculated correctly
Invalid weight	abc	Error message displayed
Health Daily routine	3	Correct output
Learn About BMI	4	Correct output
EXIT	5	Correct output

CHALLENGES FACED

During this, several challenges were encountered:

1. Understanding how to divide the program into separate Python modules.
2. Connecting different modules using Python imports.
3. Handling invalid user input without terminating the program.
4. Testing modules and functions using Jupyter Notebook.
5. Organizing the project files into a proper folder structure.
6. Setting up and uploading the project to GitHub.
7. Preparing the project documentation and diagrams.

These challenges helped improve understanding of Python programming, modular design, testing, and version control.

LEARNINGS & KEY TAKEAWAYS

Through this project, the following concepts were learned:

- Python functions and conditional statements
- Python modules and imports
- Input validation
- Exception handling
- Modular project organization
- Testing and debugging
- GitHub repository management
- Project documentation
- Creating system design diagrams
- Learned Project uploading in GitHub

FUTURE ENHANCEMENTS

Possible future improvements include:

1. Developing a graphical user interface.
2. Adding more detailed educational health information.
3. Adding optional data storage for user records.
4. Adding more comprehensive testing.
5. Improving the user interaction and presentation.
6. Adding additional educational health-related features.
7. Improving the project's documentation and visualization.
8. Converting this project into a website using applications like streamlit.

REFERENCES

1. Vidyarthi – Build Your Own Project, General Project Instructions & Submission Guidelines.
2. Python Documentation, Python programming language documentation.
3. Python BY EXAMPLE By-Nichola Lacey

